

# P0901R11

## Size feedback in operator new

Published Proposal, 2023-06-17

**This version:**

<http://wg21.link/P0901R11>

**Authors:**

[Andrew Hunter](#)

[Chris Kennelly](#) (Google)

[Thomas Köppe](#) (Google DeepMind)

**Audience:**

CWG, EWG, LWG

**Project:**

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

---

## Abstract

Provide access to actual malloc buffer sizes for users.

## Table of Contents

### 1 Motivation

- 1.1 nallocx: not as awesome as it looks
  - 1.1.1 nallocx must give a conservative answer
  - 1.1.2 nallocx duplicates work
  - 1.1.3 nallocx hides information from malloc
- 1.2 after allocation is too late
- 1.3 realloc's day has passed

### 2 Proposal

- 2.1 Allocation functions

2.2 *New-expressions*

### **3 Proposed Wording**

### **4 Implementation, Deployment and Performance Experience**

4.1 Implementability

4.2 Deployment

4.3 Performance

### **5 Alternative Designs Considered**

5.1 Parameters and return types

5.2 Why `operator new`?

5.3 New expressions

5.3.1 Which kind of "size"

### **6 Discussion**

6.1 How many `::operator new`'s?

6.2 Implementation difficulty

6.3 Interaction with Sized Delete

6.4 Advantages

6.5 Naming

### **7 Related work**

### **8 History**

8.1 R10 → R11

8.2 R9 → R10

8.3 R8 → R9

8.4 R7 → R8

8.5 R6 → R7

8.6 R5 → R6

8.7 R4 → R5

8.8 R3 → R4

8.9 R2 → R3

8.10 R1 → R2

8.11 R0 → R1

## References

### Informative References

## § 1. Motivation

Throughout this document, "malloc" refers to the **implementation** of `::operator new` both as fairly standard practice for implementers, and to make clear the distinction between the interface and the implementation.

Everyone's favorite dynamic data structure, `std::vector`, allocates memory with code that looks something like this (with many details, like `Allocator`, templating for non `char`, and exception safety, elided):

```
void vector::reserve(size_t new_cap) {
    if (capacity_ >= new_cap) return;
    const size_t bytes = new_cap;
    void *newp = ::operator new(new_cap);
    memcpy(newp, ptr_, capacity_);
    ptr_ = newp;
    capacity_ = bytes;
}
```

Consider the sequence of calls:

```
std::vector<char> v;
v.reserve(37);
// ...
v.reserve(38);
```

All reasonable implementations of malloc round sizes, both for alignment requirements and improved performance. It is extremely unlikely that malloc provided us exactly 37 bytes. We do not need to invoke the allocator here...except that we don't know that for sure, and to use the 38th byte would be undefined behavior. We would like that 38th byte to be usable without a roundtrip through the allocator.

This paper proposes an API making it safe to use that byte, and explores many of the design choices (not all of which are obvious without implementation experience.)

## § 1.1. `mallocx`: not as awesome as it looks

The simplest way to help here is to provide an informative API answering the question "If I ask for N bytes, how many do I actually get?" [\[jemalloc\]](#) and [\[TCMalloc\]](#) call this `mallocx`. We can then use that hint as a smarter parameter for operator new:

```
void vector::reserve(size_t new_cap) {
    if (capacity_ >= new_cap) return;
    const size_t bytes = mallocx(new_cap, 0);
    void *newp = ::operator new(bytes);
    memcpy(newp, ptr_, capacity_);
    ptr_ = newp;
    capacity_ = bytes;
}
```

This is a good start, and does in fact work to allow vector and friends to use the true extent of returned objects. But there are three significant problems with this approach.

### § 1.1.1. `mallocx` must give a conservative answer

While many allocators have a deterministic map from requested size to allocated size, it is by no means guaranteed that all do. Presumably they can make a reasonably good guess, but if two calls to `::operator new(37)` might return 64 and 128 bytes, we'd definitely rather know the right answer, not a conservative approximation.

### § 1.1.2. `mallocx` duplicates work

Allocation is often a crucial limit on performance. Most allocators compute the returned size of an object as part of fulfilling that allocation...but if we make a second call to `mallocx`, we duplicate all that communication, and also the overhead of the function call.

### § 1.1.3. `mallocx` hides information from malloc

The biggest problem (for the authors) is that `mallocx` discards information malloc finds valuable (the user's intended allocation size.) That is: in our running example, malloc normally knows that the user wants 37 bytes (then 38), but with `mallocx`, we will only ever be told that they want 40 (or 48, or whatever `mallocx(37)` returns.)

Google's malloc implementation ([TCMalloc]) rounds requests to one of a small (<100) number of *sizeclasses*: we maintain local caches of appropriately sized objects, and cannot do this for every possible size of object. Originally, these sizeclasses were just reasonably evenly spaced among the range they cover. Since then, we have used extensive telemetry on allocator use in the wild to tune these choices. In particular, as we know (approximately) how many objects of any given size are requested, we can solve a fairly simple optimization problem to minimize the total internal fragmentation for any choice of N sizeclasses.

Widespread use of `mallocx` breaks this. By the time TCMalloc's telemetry sees a request that was hinted by `mallocx`, to the best of our knowledge the user *wants* exactly as many bytes as we currently provide them. If a huge number of callers wanted 40 bytes but were currently getting 48, we'd lose the ability to know that and optimize for it.

Note that we can't take the same telemetry from `mallocx` calls: we have no idea how many times the resulting hint will be used (we might not allocate at all, or we might cache the result and make a million allocations guided by it.) We would also lose important information in the stack traces we collect from allocation sites.

Optimization guided by malloc telemetry has been one of our most effective tools in improving allocator performance. It is important that we fix this issue *without* losing the ground truth of what a caller of `::operator new` wants.

These three issues explain why we don't believe `mallocx` is a sufficient solution here.

## § 1.2. after allocation is too late

Another obvious suggestion is to add a way to inspect the size of an object returned by `::operator new`. Most mallocs provide a way to do this; [jemalloc] calls it `salloccx`. Vector would look like:

```
void vector::reserve(size_t new_cap) {
    if (capacity_ >= new_cap) return;
    void *newp = ::operator new(new_cap);
    const size_t bytes = salloccx(newp);
    memcpy(newp, ptr_, capacity_);
    ptr_ = newp;
    capacity_ = bytes;
}
```

This is worse than `mallocx`. It fixes the non-constant size problem, and avoids a feedback loop, but the performance issue is worse (this is the major issue *fixed* by [SizedDelete]!), and what's worse, the

above code invokes UB as soon as we touch byte `new_cap+1`. We could in principle change the standard, but this would be an implementation nightmare.

### § 1.3. `realloc`'s day has passed

We should also quickly examine why the classic C API `realloc` is insufficient.

```
void vector::reserve(size_t new_cap) {  
    if (capacity_ >= new_cap) return;  
    ptr_ = realloc(ptr_, new_cap);  
    capacity_ = new_cap;  
}
```

In principle a `realloc` from 37 to 38 bytes wouldn't carry the full cost of allocation. But it's dramatically more expensive than making no call at all. What's more, there are a number of more complicated dynamic data structures that store variable-sized chunks of data but are never actually resized. These data structures still deserve the right to use all the memory they're paying for.

Furthermore, `realloc`'s original purpose was not to allow the use of more bytes the caller already had, but to (hopefully) extend an allocation in place to adjacent free space. In a classic `malloc` implementation this would actually be possible...but most modern allocators use variants of slab allocation. Even if the 65th byte in a 64-byte allocation isn't in use, they cannot be combined into a single object; it's almost certainly required to be used for the next 64-byte allocation. In the modern world, `realloc` serves little purpose.

## § 2. Proposal

### § 2.1. Allocation functions

We propose adding new overloads of `::operator new` (so-called "size-returning allocation functions") that directly inform the user of the size available to them. C++ makes `::operator new` replaceable (15.5.4.6), allowing a program to provide its own version different from the standard library's implementation.

We note immediately that we are deliberately not providing this facility as an unrelated free function, nor leave it as an implementation detail of `std::allocator`; see [§ 5.2 Why operator new?](#) for a detailed discussion.

The new overloads are selected by a tag argument of type `std::return_size_t`, usually provided as the value `std::return_size`. (This is analogous to `std::nothrow_t/std::nothrow` and to `std::align_val_t`.)

The new overloads return a new type, `std::sized_allocation_t`, which stores a void pointer that corresponds to the pointer returned by the existing allocation functions, as well as the new size feedback information.

## § 2.2. *New-expressions*

In the previous revision, R10, we had proposed to make *new-expressions* be able to use the new allocation functions, so that a placement *new-expression* of the form `new (std::return_size) T` would have a novel return type that includes the size feedback. However, during EWG discussion it became clear that that is not a useful facility; see § 5.3 *New expressions* for details.

Consequently, we are not proposing to expose size-returning allocation functions to *new-expressions*. We propose that a hypothetical (placement) *new-expression* that would find a size-returning allocation function during lookup is ill-formed. (No such expressions can currently exist.)

## § 3. Proposed Wording

We propose wording, relative to [N4917]:

- Amend [basic.stc.dynamic.general], paragraph 2:

The library provides default definitions for the global allocation and deallocation functions. Some global allocation and deallocation functions are replaceable ([new.delete]). A C++ program shall provide at most one definition of a replaceable allocation or deallocation function. Any such function definition replaces the default version provided in the library ([replacement.functions]). The following allocation and deallocation functions ([support.dynamic]) are implicitly declared in global scope in each translation unit of a program.

```
[[nodiscard]] void* operator new(std::size_t);
[[nodiscard]] void* operator new(std::size_t, std::align_val_t);
[[nodiscard]] std::size_t operator new(std::size_t, std::align_val_t, std::return_size_t);
[[nodiscard]] std::size_t operator new(std::size_t, std::align_val_t, std::return_size_t);
```

```
void operator delete(void*) noexcept;
void operator delete(void*, std::size_t) noexcept;
void operator delete(void*, std::align_val_t) noexcept;
void operator delete(void*, std::size_t, std::align_val_t) noexcept;
```

```
[[nodiscard]] void* operator new[](std::size_t);
[[nodiscard]] void* operator new[](std::size_t, std::align_val_t);
[[nodiscard]] std::size_t operator new[](std::size_t, std::align_val_t, std::return_size_t);
[[nodiscard]] std::size_t operator new[](std::size_t, std::align_val_t, std::return_size_t);
```

```
void operator delete[](void*) noexcept;
void operator delete[](void*, std::size_t) noexcept;
void operator delete[](void*, std::align_val_t) noexcept;
void operator delete[](void*, std::size_t, std::align_val_t) noexcept;
```

These implicit declarations introduce only the function names `operator new`, `operator new[]`, `operator delete`, and `operator delete[]`.

[Note 2: The implicit declarations do not introduce the names `std`, `std::size_t`, `std::align_val_t`, `std::return_size_t`, or any other names that the library uses to declare these names. Thus, a *new-expression*, *delete-expression*, or function call that refers to one of these functions without importing or including the header `<new>` ([new.syn]) or importing a C++ library module ([std.modules]) is well-formed. However, referring to `std` or `std::size_t` or `std::align_val_t` or `std::return_size_t` is ill-formed unless a standard library declaration ([...]) of that name precedes ([basic.lookup.general]) the use of that name. — end note]

Allocation and/or deallocation functions may also be declared and defined for any class ([class.free]).

- Amend [basic.stc.dynamic.allocation], paragraph 1:

An allocation function that is not a class member function shall belong to the global scope and not have a name with internal linkage. The return type shall be `std::size_allocation_t` ([new.syn]) if the allocation function is a size-returning allocation function and `void*` otherwise. The first parameter shall have type `std::size_t` ([support.types]). The first parameter shall not have an associated default argument ([dcl.fct.default]). The value of the first parameter is interpreted as the requested size of the allocation. An allocation function is a size-returning allocation function if it has a second parameter of type `std::return_size_t`, or it has a second parameter of type `std::align_val_t` and a third parameter of type `std::return_size_t`. An allocation function can be a function template. Such a template shall declare its return type and first parameter as specified above (that is, template parameter types shall not be used in the return type and first parameter type). Allocation function templates shall have two or more parameters as specified above.

- Insert [basic.stc.dynamic.allocation], after paragraph 1:

A size-returning allocation function returns a value with a member subobject that is a pointer suitable to hold the address of potentially allocated storage, and therefore all allocation functions are said to return a pointer value.

- Amend [replacement.functions], paragraph 2:

```
operator new(std::size_t)
operator new(std::size_t, std::align_val_t)
operator new(std::size_t, const std::nothrow_t&)
operator new(std::size_t, std::align_val_t, const std::nothrow_t&)

operator new(std::size_t size, std::return_size_t)
operator new(std::size_t size, std::align_val_t al, std::return_size_t)
operator new(std::size_t size, std::return_size_t, std::nothrow_t)
operator new(std::size_t size, std::align_val_t al, std::return_size_t, std::nothrow_t)

[...]

operator new[](std::size_t)
operator new[](std::size_t, std::align_val_t)
```



```

[[nodiscard]] std::size_t operator new(
    std::size_t size, std::return_size_t);
[[nodiscard]] std::size_t operator new(
    std::size_t size, std::align_val_t alignment, std::return_size_t);
[[nodiscard]] std::size_t operator new(
    std::size_t size, std::return_size_t, std::nothrow_t) noexcept;
[[nodiscard]] std::size_t operator new(
    std::size_t size, std::align_val_t alignment, std::return_size_t,
    std::nothrow_t) noexcept;

```

[...]

```

[[nodiscard]] void* operator new[](std::size_t size);
[[nodiscard]] void* operator new[](std::size_t size, std::align_val_t align
[[nodiscard]] void* operator new[](std::size_t size, const std::nothrow_t&)
[[nodiscard]] void* operator new[](std::size_t size, std::align_val_t align
                                const std::nothrow_t&) noexcept;

```

```

[[nodiscard]] std::size_t operator new[](
    std::size_t size, std::return_size_t);
[[nodiscard]] std::size_t operator new[](
    std::size_t size, std::align_val_t alignment, std::return_size_t);
[[nodiscard]] std::size_t operator new[](
    std::size_t size, std::return_size_t, std::nothrow_t) noexcept;
[[nodiscard]] std::size_t operator new[](
    std::size_t size, std::align_val_t alignment, std::return_size_t,
    std::nothrow_t) noexcept;

```

[...]

```

[[nodiscard]] void* operator new (std::size_t size, void* ptr) noexcept;
[[nodiscard]] void* operator new[](std::size_t size, void* ptr) noexcept;
void operator delete (void* ptr, void*) noexcept;
void operator delete[](void* ptr, void*) noexcept;

```

- Insert [new.delete.single], after paragraph 4:

```
[[nodiscard]] std::size_t allocation_t::operator new(  
    std::size_t size, std::return_size_t);  
[[nodiscard]] std::size_t allocation_t::operator new(  
    std::size_t size, std::align_val_t alignment, std::return_size_t);
```

- Effects: Same as above, except these are called by a placement version of a *new-expression* when a C++ program prefers the size-returning allocation function.
- Replaceable: A C++ program may define functions with either of these function signatures, and thereby displace the default versions defined by the C++ standard library.
- Required behavior: Return a `sized_allocation_t` whose `ptr` member represents the address of a region of *N* bytes of suitably aligned storage ([basic.stc.dynamic]) for some  $N \geq \text{size}$ , and whose `bytes` member is *N*, or else throw a `bad_alloc` exception. This requirement is binding on any replacement versions of these functions.
- Default behavior: Returns `std::size_t allocation_t{operator new(size), N}` and `std::size_t allocation_t{operator new(size, alignment), N}` respectively. If a user-provided operator `new` is invoked directly or indirectly, *N* is `size`.

- Insert [new.delete.single], after paragraph 9:

```
[[nodiscard]] std::size_t operator new(
    std::size_t size, std::return_size_t, std::nothrow_t) noexcept;
[[nodiscard]] std::size_t operator new(
    std::size_t size, std::align_val_t alignment, std::return_size_t, std::
```

- Effects: Same as above, except these are called by a placement version of a *new-expression* when a C++ program prefers the size-returning allocation function and a null pointer result as an error indication.
- Replaceable: A C++ program may define functions with either of these function signatures, and thereby displace the default versions defined by the C++ standard library.
- Required behavior: Return a `sized_allocation_t` whose `ptr` member represents the address of a region of *N* bytes of suitably aligned storage ([basic.stc.dynamic]) for some *N*  $\geq$  *size*, and whose `bytes` member is *N*, or else return `std::size_t{nullptr, 0}`. Each of these `nothrow` versions of `operator new` returns a pointer obtained as if acquired from the (possibly replaced) corresponding non-placement function. This requirement is binding on any replacement versions of these functions.
- Default behavior: Returns `std::size_t{operator new(size), N}` and `std::size_t{operator new(size, alignment), N}` respectively. If a user-provided `operator new` is invoked directly or indirectly, *N* is *size*. If the call to `operator new` throws, returns `std::size_t{nullptr, 0}`.

- Amend [new.delete.single], paragraph 11:

```
void operator delete(void* ptr) noexcept;
void operator delete(void* ptr, std::size_t size) noexcept;
void operator delete(void* ptr, std::align_val_t alignment) noexcept;
void operator delete(void* ptr, std::size_t size, std::align_val_t alignmer
```

[...]

- If the `alignment` parameter is not present, `ptr` shall have been returned by an allocation function without an alignment parameter. If present, the `alignment` argument shall equal the alignment argument passed to the allocation function that returned `ptr`. If present, the `size` argument shall equal the `size` argument passed to the allocation function that returned `ptr`, if

ptr was not allocated by a size-returning allocation function. If present, the size argument shall satisfy bytes  $\geq$  size  $\geq$  requested if ptr was allocated by a size-returning allocation function, where bytes is the size returned in std::sized\_allocation\_t and requested is the size argument passed to the allocation function.

[...]

- Insert [new.delete.array], after paragraph 4:

```
[[nodiscard]] std::sized_allocation_t ::operator new[](
    std::size_t size, std::return_size_t);
[[nodiscard]] std::sized_allocation_t ::operator new[](
    std::size_t size, std::align_val_t alignment, std::return_size_t);
```

- *Effects:* Same as above, except these are called by a placement version of a *new-expression* when a C++ program prefers the size-returning allocation function.
- *Replaceable:* A C++ program may define functions with either of these function signatures, and thereby displace the default versions defined by the C++ standard library.
- *Required behavior:* Return a sized\_allocation\_t whose ptr member represents the address of a region of *N* bytes of suitably aligned storage ([basic.stc.dynamic]) for some  $N \geq$  size, and whose bytes member is *N*, or else throw a bad\_alloc exception. This requirement is binding on any replacement versions of these functions.
- *Default behavior:* Returns std::sized\_allocation\_t{operator new[](size), N} and std::sized\_allocation\_t{operator new[](size, alignment), N} respectively. If a user-provided operator new is invoked directly or indirectly, *N* is size.

- Insert [new.delete.array], after paragraph 8:

```
[[nodiscard]] std::size_t allocation_t::operator new[](
    std::size_t size, std::return_size_t, std::nothrow_t) noexcept;
[[nodiscard]] std::size_t allocation_t::operator new[](
    std::size_t size, std::align_val_t alignment, std::return_size_t, std::
```

- Effects: Same as above, except these are called by a placement version of a *new-expression* when a C++ program prefers the size-returning allocation function and a null pointer result as an error indication.
- Replaceable: A C++ program may define functions with either of these function signatures, and thereby displace the default versions defined by the C++ standard library.
- Required behavior: Return a `sized_allocation_t` whose `ptr` member represents the address of a region of *N* bytes of suitably aligned storage ([basic.stc.dynamic]) for some *N*  $\geq$  *size*, and whose `bytes` member is *N*, or else return `std::size_t allocation_t{nullptr, 0}`. This requirement is binding on any replacement versions of these functions.
- Default behavior: Returns `std::size_t allocation_t{operator new[](size), N}` and `std::size_t allocation_t{operator new[](size, alignment), N}` respectively. If a user-provided operator `new` is invoked directly or indirectly, *N* is *size*. If the call to operator `new[]` throws, returns `std::size_t allocation_t{nullptr, 0}`.

- Amend [new.delete.array], paragraph 10:

```
void operator delete[](void* ptr) noexcept;
void operator delete[](void* ptr, std::size_t size) noexcept;
void operator delete[](void* ptr, std::align_val_t alignment) noexcept;
void operator delete[](void* ptr, std::size_t size, std::align_val_t alignm
```

[...]

- If the `alignment` parameter is not present, `ptr` shall have been returned by an allocation function without an `alignment` parameter. If present, the `alignment` argument shall equal the `alignment` argument passed to the allocation function that returned `ptr`. If present, the `size` argument shall equal the `size` argument passed to the allocation function that returned `ptr`, if `ptr` was not allocated by a size-returning allocation function. If present, the `size` argument shall satisfy *bytes*  $\geq$  *size*  $\geq$  *requested* if `ptr` was allocated by a size-returning allocation

function, where `bytes` is the size returned in `std::size_allocation_t` and `requested` is the size argument passed to the allocation function.

[...]

- Amend Table 19 ("Feature-test macros") [cpp.predefined]

| Name                                  | Value            |
|---------------------------------------|------------------|
| <code>__cpp_size_returning_new</code> | PLACEHOLDER DATE |

## § 4. Implementation, Deployment and Performance Experience

### § 4.1. Implementability

The fundamental size feedback logic is a shipping part of TCMalloc ([\[MallocExtension\]](#)), as a set of functions such as:

```
tcmalloc::size_ptr_t tcmalloc_size_returning_operator_new(std::size_t size
```

We implemented the new `::operator new` overload in Clang. This presented no noteworthy complication or obstacle. An `::operator new` whose return type is not `void*` is novel, but entirely implementable.

Building a few large applications from Google's codebase with the modified version of Clang showed nothing unexpected and continued working.

### § 4.2. Deployment

We modified the libc++ standard library implementation with minimal effort to use that function. (The implementation has a convenient [central point](#) at which the underlying allocation function is called.) We were able to build a number of significant production binaries of the Google codebase without problems.

As above, building a few large applications from Google's codebase succeeded without incident.

## § 4.3. Performance

At Google, we modified libc++'s `basic_string` to use TCMalloc's size-returning allocation function. This resulted in savings on heap allocations that we will describe below. In summary, the proposed feature allows vectors and strings to grow more efficiently. While it is hard to spot these effects in real-world benchmarks, where they are within the noise, we performed micro-benchmarks and traced allocations to show that string allocations are reduced by about 4%. For the application deployment in Google's production systems, we extrapolate fleet-wide memory savings from strings of about 0.2%.

One source of inefficiencies is the interplay between malloc and the implementation of `basic_string`. The libc++ implementation of `basic_string` always allocates memory in multiples of 16 bytes. The glibc malloc (ptmalloc2) allocates blocks of memory of size  $24 + n \times 16$ . For example, a call `reserve(35)` would make `basic_string` allocate a rounded-up value of 48, but malloc would allocate 56 bytes. With size feedback, `basic_string` would ask for "at least 35 bytes" and receive 40 bytes from malloc. With TCMalloc the situation is different, but savings are still possible: its size classes are 8, 16, 32, 48, 64, followed by 8-byte increments up to 128. In this case, size feedback would allow `basic_string` to hit every size bucket instead of only every even one for strings of length 64–128.

TCMalloc's size feedback is used in Google's implementation of `absl::Cord` and of `google::protobuf::RepeatedField`.

## § 5. Alternative Designs Considered

### § 5.1. Parameters and return types

Another signature we could use would be:

```
enum class return_size_t : std::size_t {};  
void* ::operator new(std::size_t size, std::return_size_t&);
```

(and so on.) This is slightly simpler to read as a signature, but arguably worse in usage:

```
std::tie(obj.ptr, obj.size) = ::operator new(37, std::return_size_t{});  
  
// ...vs...  
  
// Presumably the object implementation wants to contain a size_t,
```

```
// not a return_size_t.  
std::return_size_t rs;  
obj.ptr = ::operator new(37, rs);  
obj.size = rs;
```

More importantly, this form is less efficient. In practice, underlying malloc implementations provide actual definitions of `::operator new` symbols which are called like any other function. Passing a reference parameter requires us to actually return the size via memory.

- Linux ABIs support returning at least two scalar values in registers (even if they're members of a trivially copyable struct) which can be dramatically more efficient.
- The [\[MicrosoftABI\]](#) returns large types by pointer, but this is no worse than making the reference parameter an inherent part of the API.

Whether we use a reference parameter or a second returned value, the interpretation is the same.

## § 5.2. Why `operator new`?

Given the added complexity of the `operator new` overload set, one might ask why we want to expose this facility as such an overload, rather than something else. Concretely, we could provide a completely unrelated free function, or we could ask that the new behavior be exposed via `std::allocator<T>::allocate_at_least`.

Those alternatives fall short, however, since the `operator new` allocation functions have a special place in C++ that the alternatives do not have: They are *replaceable* and can be chosen by the application owner:

- Compared to an unrelated free function: an unrelated free function would not be used by existing code, in particular, code that may be outside the user's control. The replaceable allocation function, by contrast, can affect and benefit existing code. Moreover, it would require additional integration to make the free function interoperate with `operator delete`, whereas we have an existing framework for matching `operator new` and `operator delete`. Finally, `operator new` enjoys various special properties regarding implicit lifetimes which would need to be added explicitly to an unrelated free function.
- Compared to `std::allocator`: The implementation of `std::allocator` is generally under the control of the standard library vendor, not the user. The vendor is not going to select any one particular malloc implementation in their `std::allocator`. In fact, the user may want to use different malloc implementations in different projects. By contrast, a replaceable allocation function opens the choice of malloc implementation to the user on a program-by-program basis.

(Note that `std::allocator` is specified to use `::operator new`, and while no further details (such as choice of overload) are specified, it is reasonable to assume that a size-returning `operator new` would be picked up by `std::allocator::allocate_at_least` as a matter of quality-of-implementation.)

Finally, a suggestion that came up during the review of R10 was that we could aim at a *lower* level and propose a new `malloc`-like facility to both C and C++. While this would undoubtedly be useful, we believe that it does not replace `operator new`. That is because `operator new` is generally how one "obtains memory in C++"; it enjoys special properties in the language, e.g. regarding implicit lifetimes, and also with regards to elision or merging of allocations.

As a data point, the codebase at Google uses TCMalloc and its `operator new` interface, not its `malloc` interface, which are entirely different. A significant performance gain comes from the use of `operator new` and sized-`operator delete`, compared to the `malloc/free` interface. (The vast majority of memory allocations in that codebase come from `operator new` and not from `malloc`.) Exposing the size feedback as part of this established framework makes the performance benefits available in ways that a less integrated facility would not easily be able to do.

### § 5.3. New expressions

We considered previously to have "size-returning *new-expressions* that expose the excess allocation space resulting from the allocation that's done as part of the *new-expression*. However, it is exceedingly awkward to make use of the excess allocation space, since the lifetime of the object(s) created by the `new-expression` would be distinct from whatever objects one later creates manually in the excess space.

The following example was given in the previous revision:

```
auto [p, sz] = new (std::return_size) T[5];
for (int i = 5; i < sz / sizeof(T); i++) {
    new (p[i]) T;
}
for (int i = 0; i < sz / sizeof(T); i++) {
    p[i].DoStuff();
}
for (int i = 5; i < sz / sizeof(T); i++) {
    p[i].~T();
}
delete[] p;
```

The example shows how the lifetime of the "requested" objects is managed by the delete expression, but the lifetime of the manually constructed excess objects has to be handled manually. This is awkward, easy to misuse, and we do not see any need for such a facility.

Non-array *new-expressions* would be similarly useless.

One conceivable use case, which we ultimately will not pursue, is perhaps worth describing: for trivial types, such as `char`, one could imagine a chain of expressions `new (std::return_size) char[N]`. These could successively return excess storage, but in addition, the compiler would be allowed to merge the allocations, because of the special allowances made to how *new-expressions* (don't) call allocation functions. This behavior is not achievable with `operator new` function calls alone.

We also note in passing that in R10 `std::sized_allocation_t` was a template, whose template parameter would take the type used in the *new-expression*. There were implementer objections to having a core language facility having to instantiate templates.

### § 5.3.1. Which kind of "size"

We considered alternatives for returning the size.

- We could return two pointers, the initial object and one past the end of the array (minus the array allocation overhead).

```
auto [start, end] = new (std::return_size) T[5];
for (T* p = start + 5; p != end; p++) {
    new (p) T;
}
for (T* p = start; p != end; p++) {
    p->DoStuff();
}
for (T* p = start + 5; p != end; p++) {
    p->~T();
}
delete[] start;
```

The pair of pointers provides convenience for use with iterator-oriented algorithms. The problem we foresee is that a size-returning allocation function may not provide a size that is an appropriate multiple of `sizeof(T)` (or fail to be a multiple of `alignof(T)`), a possible, but unlikely

scenario). This imposes a need for more extensive logic around the *new-expression* in handling the returned allocation size.

- We could return the size in units of `T`, this leads to an inconsistency between the expected usage for `new` and `new[]`:
  - For `new`, we may only end up fitting a single `T` into an allocator size quanta, so the extra space remains unusable. If we can fit multiple `T` into a single allocator size quanta, we now have an array from what was a scalar allocation site. This cannot be foreseen by the compiler as `::operator new` is a replaceable function.
  - For `new[]`, the size in units of `T` can easily be derived from the returned size in bytes.
- We could pass the size in units of `T` or bytes to the constructor of `T`:
  - For `new`, this is especially useful for tail-padded arrays, but neglects default-initialized `T`.
  - For `new[]`, a common use case is expected to be the allocation of arrays of `char`, `int`, etc. The size of the overall array is irrelevant for the individual elements.
- We could return the size via a reference parameter:

```
std::return_end<T> end;
T* p = new (end) T[5];
for (T* p = start + 5; p != end; p++) {
    new (p) T;
}
for (T* p = start; p != end; p++) {
    p->DoStuff();
}
for (T* p = start + 5; p != end; p++) {
    p->~T();
}
```

or, demonstrated with bytes:

```
std::return_size_t size;
T* p = new (size) T[5];
for (int i = 5; i < size / sizeof(T); i++) {
    new (p[i]) T;
}
for (int i = 0; i < size / sizeof(T); i++) {
    p[i].DoStuff();
}
```

```
for (int i = 5; i < size / sizeof(T); i++) {  
    p[i].~T();  
}  
delete[] p;
```

(Casts omitted for clarity.)

As discussed for `::operator new` in § 3 [Proposed Wording](#), a reference parameter poses difficulties for optimizers and involves returning the size via memory (depending on ABI).

For `new[]` expressions, we considered alternatively initializing the returned `(sz / sizeof(T))` number of elements.

- This would avoid the need to explicitly construct / destruct the elements with the additional returned space (if any).

The *new-initializer* is invoked for the returned number of elements, rather than the requested number of elements. This allows `delete[]` to destroy the correct number of elements (by storing `sz / sizeof(T)` in the array allocation overhead).

- The presented proposal (leaving this space uninitialized) was chosen for consistency with `new`.

## § 6. Discussion

### § 6.1. How many `::operator new`'s?

It is unfortunate that we have so many permutations of `::operator new`--eight seems like far more than we should really need! But there really isn't any significant runtime cost for having them. Use of raw calls to `::operator new` is relatively rare: It's a *building block* for low-level libraries, allocators ([\[P0401R4\]](#)), and so on, so the cognitive burden on C++ users is low.

The authors have considered other alternatives to the additional overloads. At the Jacksonville meeting, EWG suggested looking at parameter packs.

- Parameter packs do not reduce the number of symbols introduced. Implementers still need to provide implementations each of the *n* overloads.
- Retrofitting parameter packs leaves us with *more* mangled variants. Implementers need to provide both the legacy symbols as well as the parameter pack-mangled symbols.

The authors have also considered APIs where all parameters are passed, thereby requiring a single new overload. This adds further overhead for implementations, as it moves compile-time decisions (is the

alignment at or below the minimum guaranteed by `operator new`) into runtime ones.

The alternative to modifying the handling of *new-expressions* invoking deallocation functions (when an exception is thrown) would require additional overloads for `operator delete` / `operator delete[]` whose sole purpose would be to accept and discard the `std::return_size_t`.

## § 6.2. Implementation difficulty

It's worth reiterating that there's a perfectly good trivial implementation of these functions:

```
std::size_allocation_t ::operator new(std::size_t n, std::return_size_t) {  
    return {::operator new(n), n};  
}
```

Malloc implementations are free to properly override this with a more impactful definition, but this paper poses no significant difficulty for toolchain implementers.

Implementation Experience:

- TCMalloc has developed an implementation opensourced on GitHub ([\[MallocExtension\]](#)). While this requires mapping from an integer size class to the true number of bytes, combining this lookup with the allocation is more efficient as we avoid recomputing the sizeclass itself (given a request) or deriving it from the object's address.
- jemalloc is prototyping a `smallocx` function providing a C API for this functionality [\[smallocx\]](#).

## § 6.3. Interaction with Sized Delete

For allocations made with `sized_allocation_t`-returning `::operator new`, we need to relax `::operator delete`'s size argument (16.6.2.1 and 16.6.2.2). For allocations of `T`, the size quanta used by the allocator may not be a multiple of `sizeof(T)`, leading to both the original and returned sizes being unrecoverable at the time of deletion.

Consider the memory allocated by:

```
using T = std::aligned_storage<16, 8>::type;  
  
std::vector<T> v(4);
```

The underlying heap allocation is made with `::operator new(64, std::return_size_t)`.

- The memory allocator may return a 72 byte object: Since there is no `k` such that `sizeof(T) * k = 72`, we can't provide that value to `::operator delete(void*, size_t)`. The only option would be storing 72 explicitly, which would be wasteful.
- The memory allocator may instead return an 80 byte object (5 `T`'s): We now cannot represent the original request when deallocating without additional storage.

For allocations made with

```
std::tie(p, m) = ::operator new(n, std::return_size_t{});
```

we permit `::operator delete(p, s)` where  $n \leq s \leq m$ .

This behavior is consistent with `[jemalloc]`'s `sdallocx`, where the deallocation size must fall between the request (`n`) and the actual allocated size (`m`) inclusive. It is also consistent with the standard library's allocator interface (as amended by [\[P0401R4\]](#)).

## § 6.4. Advantages

It's easy to see that this approach nicely solves the problems posed by other methods:

- We pay almost nothing in speed to return an actual-size parameter. For `TCMalloc` and `jemalloc`, this is typically a load from to map from `sizeclass` to `size`. This cost is strictly smaller than with `naallocx` or the like, as that same translation must be performed in addition to the duplicative work previously discussed.
- We are told exactly the size we have, without risk of UB. We can avoid subsequent reallocations when growing to a buffer to an already-allocated size.
- Allocator telemetry knows actual request sizes exactly.

## § 6.5. Naming

The library support type is named `sized_allocation_t`, based on LEWG's naming suggestions and for consistency (in choice of `_t`) with the other allocation library support types (`size_t`, `align_val_t`, `nothrow_t`, etc.). We expect this to be spelled rarely.

Its members are `ptr` and `bytes`. `ptr` for the memory returned is an intuitive name. `bytes` conveys units in its name (`bytes`). Since new expressions can return additional storage under this proposal,

this distinguishes it from returning the amount of storage available for *whole* objects. This contrasts with [P0401R4], which does return storage for whole objects and whose support type's field is named `count`.

## § 7. Related work

[P0401R4] considers this problem at the level of the `Allocator` concept. Ironically, the lack of the above API was one significant problem in its original revision: how could an implementation of `std::allocator` provide the requested feedback in a way that would work with any underlying malloc implementation? See also § 5.2 [Why operator new?](#).

## § 8. History

### § 8.1. R10 → R11

Support for *new-expressions* is removed, and the class template `sized_return_t` is changed to a non-template class with a `void*` member.

Rationale added why the facility needs to be provided as an allocation function, rather than some unrelated free function or as a detail of `std::allocator`.

Reports on implementation and deployment experience and impact on runtime allocations have been added.

### § 8.2. R9 → R10

A detailed design of *new-expressions* has been added. Previously, changes to *new-expressions* had only been outlined at a high level. As a result, previously suggested changes to overload resolution rules for finding the allocation function (which was allowed to fall back to a non-size-returning allocation function, with unclear consequences) have been removed. The relation between *new-expressions* and allocation functions is now described explicitly.

Sections are reorganized: alternative designs are now separate from a discussion of the chosen design, and an expanded section on *new-expressions* has been moved up before the proposed wording.

### § 8.3. R8 → R9

CWG reviewed [P0901R8] via [CWG2021Telecon].

Wording changes have been made to the proposal based on CWG feedback and assistance from Ryan McDougall.

### § 8.4. R7 → R8

LEWG reviewed [P0901R6] via telecon.

Poll: Send P0901R6, after changing `p` to `ptr` and `n` to `bytes` in `sized_allocation_t`, to electronic ballot to be forwarded to CWG.

| SF | F  | N | A | SA |
|----|----|---|---|----|
| 4  | 11 | 0 | 0 | 0  |

- Applied member name feedback from LEWG to rename `sized_allocation_t`'s members from `p` to `ptr` and `n` to `bytes`. Added naming rationale section.
- Update reference to revised [P0401R4].
- Refined wording in proposal.

### § 8.5. R6 → R7

- Removed extraneous `std::` typo

### § 8.6. R5 → R6

LEWG reviewed [P0901R5] library support types at [Prague].

- LEWG discussed the similarity to `std::span`. Participants in the discussion were concerned about the potential library layering issue (`<new>` layers below practically everything) and that ownership is conveyed in the return value. In contrast, `std::span` is non-owning.
- LEWG discussed whether the type needed a name. It needs to be named to allow users to replace `operator new` and spell the type.

LEWG took an approval poll of different names

|    |                     |
|----|---------------------|
| 5  | sized_ptr_t         |
| 2  | sized_ptr           |
| 3  | memblock_t          |
| 7  | memblock            |
| 0  | memspan_t           |
| 2  | memspan             |
| 8  | alloc_size_t        |
| 4  | alloc_size          |
| 5  | alloc_t             |
| 0  | alloc               |
| 13 | sized_allocation_t  |
| 17 | sized_allocation    |
| 0  | alloc_span_t        |
| 2  | alloc_span          |
| 9  | allocation_t        |
| 9  | allocation          |
| 16 | alloc_result_t      |
| 14 | alloc_result        |
| 12 | allocation_result_t |
| 16 | allocation_result   |

Based on this poll, LEWG directed the paper authors to consider `sized_allocation`, `alloc_result_t`, and `allocation_result`.

LEWG additionally polled on whether the type should have a `_t` suffix.

| SF | F | N | A | SA |
|----|---|---|---|----|
| 0  | 6 | 7 | 8 | 1  |

Based on this feedback, the authors have chosen `sized_allocation_t`, based on LEWG's naming suggestions and for consistency (in choice of `_t`) with the other allocation library support types (`size_t`, `align_val_t`, `nothrow_t`, etc.). We expect this to be spelled rarely.

## § 8.7. R4 → R5

EWG reviewed P0901R4 at [\[Cologne\]](#).

Poll: P0901R4 as presented, forward to LEWG for C++23, *not* C++20.

| SF | F  | N  | A | SA |
|----|----|----|---|----|
| 2  | 11 | 14 | 2 | 0  |

- Fixed typos in examples.
- Added proposed feature test macro.
- Added LEWG audience for library support type names.

## § 8.8. R3 → R4

- Update reference to revised [\[P0401R1\]](#).

## § 8.9. R2 → R3

- Added proposed wording.
- For newly added allocation functions, `std::nothrow_t` is passed by value, rather than reference.

## § 8.10. R1 → R2

Applied feedback from San Diego Mailing

- Moved from passing `std::return_size_t` parameter by reference to by value. For many ABIs, this is more optimizable and to the authors' knowledge, no worse on any other.
- Added rationale for not using parameter packs for this functionality.

## § 8.11. R0 → R1

Applied feedback from [\[JacksonvilleMinutes\]](#).

- Clarified in [§ 3 Proposed Wording](#) the desire to leverage the existing "replacement functions" wording of the IS, particularly given the close interoperation with the existing `::operator new/::operator delete` implementations.
- Added a discussion of the Microsoft ABI in [§ 3 Proposed Wording](#).

- Noted in [§ 6.1 How many ::operator new's?](#) the possibility of using a parameter pack.
- Added a proposal for [§ 2.2 New-expressions](#), as requested by EWG.

Additionally, a discussion of [§ 6.3 Interaction with Sized Delete](#) has been added.

## § References

### § Informative References

#### [Cologne]

*Size feedback in operator new*. 2019-09-17.

#### [CWG2021Telecon]

*Paper Review Teleconference 2021-03-22*. 2021-03-22.

#### [JacksonvilleMinutes]

*Jacksonville 2018 minutes*. 2018-03-15.

#### [JEMALLOC]

*jemalloc(3) - Linux man page*. URL: <http://jemalloc.net/jemalloc.3.html>

#### [MallocExtension]

*TCMalloc Malloc Extensions*. URL:  
[https://github.com/google/tcmalloc/blob/master/tcmalloc/malloc\\_extension.h](https://github.com/google/tcmalloc/blob/master/tcmalloc/malloc_extension.h)

#### [MicrosoftABI]

*Return Values*. 2016-11-03. URL: <https://docs.microsoft.com/en-us/cpp/build/return-values-cpp>

#### [N4917]

*Working Draft, Standard for Programming Language C++*. 2022-09-05. URL: <https://open-std.org/jtc1/sc22/wg21/docs/papers/2022/n4917.pdf>

#### [P0401R1]

Jonathan Wakely; Chris Kennelly. *Providing size feedback in the Allocator interface*. 2019-06-11. URL: <http://wg21.link/P0401R1>

#### [P0401R4]

Jonathan Wakely; Chris Kennelly. *Providing size feedback in the Allocator interface*. 2020-11-14. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p0401r4.html>

#### [P0901R5]

*Size feedback in operator new*. 2019-10-06. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0901r5.html>

#### [P0901R6]

*Size feedback in operator new*. 2020-03-01. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p0901r6.html>

**[P0901R8]**

Chris Kennelly, Andrew Hunter. *Size feedback in operator new*. 15 December 2020. URL: <https://wg21.link/p0901r8>

**[Prague]**

*LEWG Prague Minutes for P901R5*. 2020-02-10.

**[SizedDelete]**

Lawrence Crowl. *C++ Sized Deallocation*. URL: <http://wg21.link/n3536>

**[SMALLOCX]**

*Add experimental API to support P0901r0*. URL: <https://github.com/jemalloc/jemalloc/pull/1270>

**[TCMalloc]**

*TCMalloc*. URL: <https://github.com/google/tcmalloc>